

Proyecto DevOps - Aplicación Web Segura en Kubernetes con Monitoreo Activo

Valentina Rodriguez y Santiago Santafe

Descripción del proyecto

Este proyecto corresponde a la entrega final y consolidada del laboratorio técnico de DevOps (Metodología ABP).

El objetivo principal es la implementación de un pipeline CI/CD completamente funcional que integra de extremo a extremo prácticas de seguridad automatizada (**DevSecOps**) y observabilidad en tiempo real (**Monitoreo**), garantizando un ciclo de vida de desarrollo de software eficiente y seguro.

La aplicación está compuesta por:

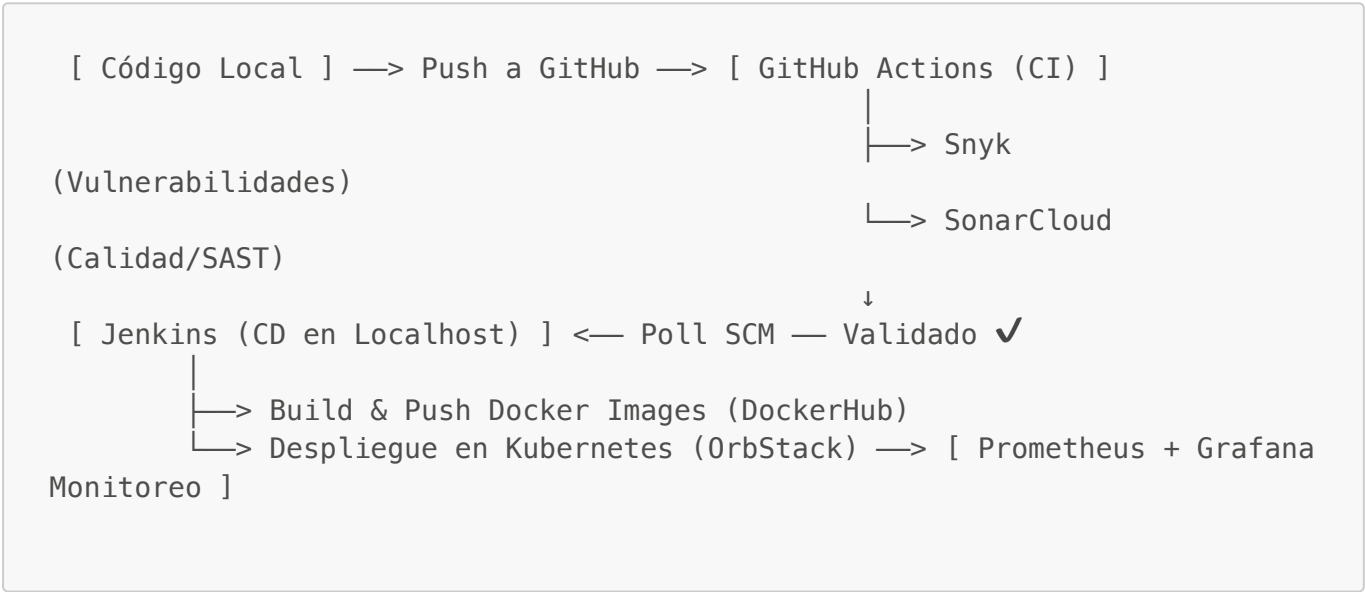
- **Frontend:** Desarrollado con React y Vite.
- **Backend:** Desarrollado con Node.js y Express.
- **Base de Datos:** MongoDB.
- **Contenedores:** Docker de arquitectura múltiple.
- **Orquestación:** Manifiestos de Kubernetes ejecutados sobre clúster nativo local.
- **Pipeline CI:** Automatizado en GitHub Actions con escaneos de código y dependencias.
- **Pipeline CD:** Definido y orquestado mediante Jenkins en bloques de entrega continua.
- **Monitoreo:** Recolección y visualización de métricas nativas del clúster.

Tecnologías utilizadas y Justificación

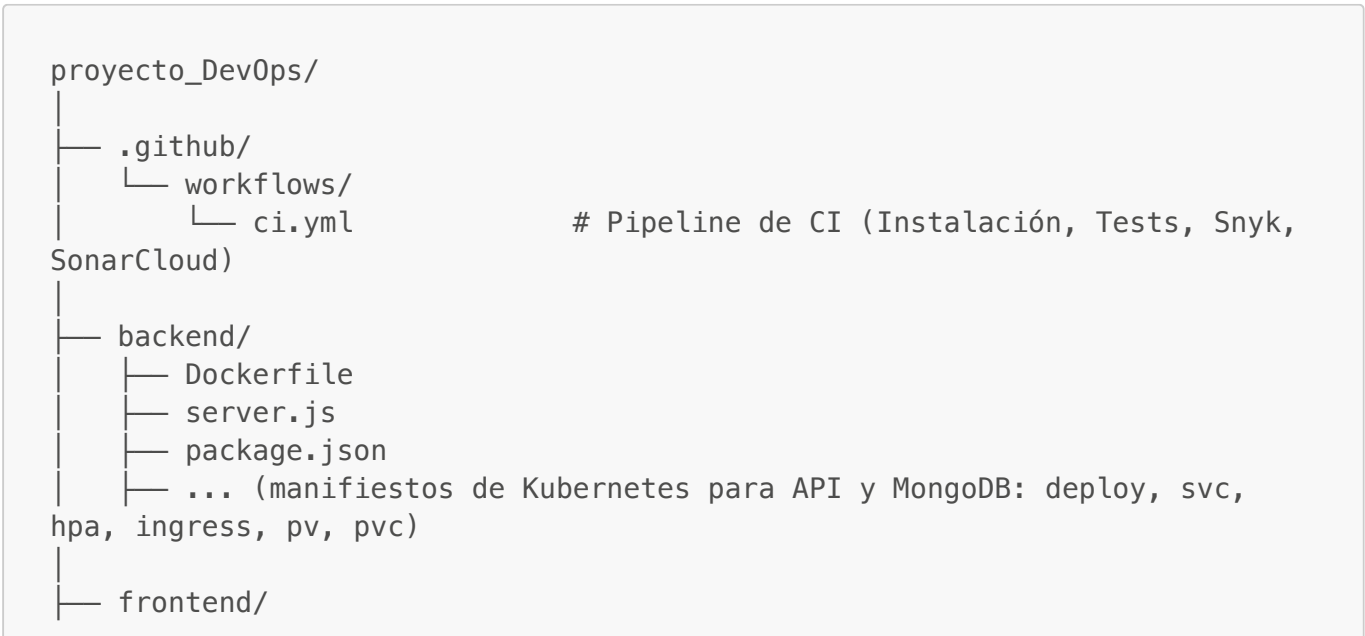
Tecnología	Uso dentro del proyecto	Justificación Técnica
GitHub	Alojamiento del código fuente	Repositorio centralizado para el control de versiones y disparador de eventos CI.
GitHub Actions	Automatización de Integración Continua (CI)	Ejecuta flujos de trabajo aislados en la nube ante cada push, optimizando el tiempo de feedback.
Jenkins	Pipeline de Entrega Continua (CD)	Orquesta de manera flexible las etapas de empaquetado, validación y despliegue local.
SonarCloud	Análisis Estático de Código (SAST)	Versión SaaS de SonarQube. Evalúa la mantenibilidad, duplicación y encuentra fallos de seguridad sin consumir recursos del clúster.
Snyk	Escaneo de Dependencias (SCA)	Detecta vulnerabilidades y exploits conocidos en librerías de terceros dentro de los <code>package.json</code> .
Docker / DockerHub	Contenerización y Registro	Empaqueta la aplicación en imágenes portables e inmutables listas para producción.

Tecnología	Uso dentro del proyecto	Justificación Técnica
Kubernetes (OrbStack)	Orquestación de Contenedores	Entorno de ejecución de alta eficiencia que gestiona el ciclo de vida, escalado y red de los Pods.
Helm	Gestor de Paquetes para K8s	Permite instalar y versionar arquitecturas complejas (como la pila de monitoreo) con un solo comando.
Prometheus	Recolección de Métricas	Base de datos de series temporales que extrae automáticamente métricas de estado del clúster.
Grafana	Visualización de Datos	Panel de control interactivo para monitorizar CPU, memoria y rendimiento general en tiempo real.

Arquitectura del Flujo CI/CD/SecOps



Estructura Final del Repositorio



```
|   |   | Dockerfile
|   |   | src/
|   |   | ... (manifiestos de Kubernetes para Frontend: deploy, svc, ingress)
|   |   |
|   |   | monitoring/
|   |   |   | README.md # Instrucciones del despliegue del stack de
|   |   |   | monitoreo Helm
|   |   |
|   |   | Jenkinsfile # Pipeline de CD completamente estructurado
|   |   | README.md
```

Pipeline CI - GitHub Actions (Con Seguridad Integrada)

El pipeline de integración continua (**ci.yml**) automatiza la validación de calidad y la seguridad del código fuente antes de permitir cualquier despliegue.

Etapas del pipeline CI:

1. **Checkout:** Descarga el código con historial profundo para trazabilidad.
2. **Setup Node.js:** Prepara el entorno de ejecución según la versión del proyecto.
3. **Snyk Security Scan:** Analiza de forma estricta las dependencias en busca de paquetes vulnerables.
4. **Install & Test:** Descarga las dependencias e instala componentes pasando los test unitarios.
5. **SonarCloud Analysis:** Ejecuta el motor de análisis estático buscando malas prácticas y code smells.

Pipeline CD - Jenkins

El archivo **Jenkinsfile** define un pipeline declarativo funcional estructurado en fases limpias y diseñado para entornos operativos de alta disponibilidad:

1. **Checkout Repository:** Sincroniza la última versión del código validado en el CI.
2. **Build Docker Images:** Compila las imágenes del frontend y el backend de forma aislada.
3. **Security Verification:** Valida que las firmas de calidad pasadas en la nube se encuentren aprobadas.
4. **Push to Registry:** Publica las imágenes inmutables etiquetadas en DockerHub.
5. **Deploy to Kubernetes Cluster:** Aplica los manifiestos YAML actualizando los deployments en el clúster.
6. **Verify Prometheus Monitoring:** Certifica que Prometheus descubra los nuevos servicios para iniciar el rastreo.

Evidencias de Ejecución y Resultados

1. Evidencias de Integración Continua (CI)

Captura del Pipeline Completo de GitHub Actions

Muestra la ejecución exitosa de los Jobs concurrentes de Frontend y Backend.

Summary

All jobs

Backend CI & Security

Frontend CI & Security

Run details

Usage

Workflow file

Backend CI & Security

failed 1 minute ago in 33s

Search logs

Fix with Copilot

Explain error

Checkout repository

Setup Node.js

Install backend dependencies

Run Snyk to check Backend vulnerabilities

1 ▶ Run snyk/actions/node@master

8 /usr/bin/docker run --name snyksnyknode_0061fd --label de9844 --workdir /github/workspace --rm -e "SNYK_TOKEN" -e "INPUT_ARGS" -e "INPUT_COMMAND" -e "INPUT_JSON" -e "FORCE_COLOR" -e "SNYK_INTEGRATION_NAME" -e "SNYK_INTEGRATION_VERSION" -e "HOME" -e "GITHUB_JOB" -e "GITHUB_REF" -e "GITHUB_SHA" -e "GITHUB_REPOSITORY" -e "GITHUB_REPOSITORY_OWNER" -e "GITHUB_REPOSITORY_OWNER_ID" -e "GITHUB_RUN_ID" -e "GITHUB_RUN_NUMBER" -e "GITHUB_RETENTION_DAYS" -e "GITHUB_RUN_ATTEMPT" -e "GITHUB_ACTOR_ID" -e "GITHUB_ACTOR" -e "GITHUB_WORKFLOW" -e "GITHUB_HEAD_REF" -e "GITHUB_BASE_REF" -e "GITHUB_EVENT_NAME" -e "GITHUB_SERVER_URL" -e "GITHUB_API_URL" -e "GITHUB_GRAPHQL_URL" -e "GITHUB_REF_NAME" -e "GITHUB_REF_PROTECTED" -e "GITHUB_REF_TYPE" -e "GITHUB_WORKFLOW_REF" -e "GITHUB_WORKFLOW_SHA" -e "GITHUB_REPOSITORY_ID" -e "GITHUB_TRIGGERING_ACTOR" -e "GITHUB_WORKSPACE" -e "GITHUB_ACTION" -e "GITHUB_EVENT_PATH" -e "GITHUB_ACTION_REPOSITORY" -e "GITHUB_ACTION_REF" -e "GITHUB_PATH" -e "GITHUB_ENV" -e "GITHUB_STEP_SUMMARY" -e "GITHUB_STATE" -e "GITHUB_OUTPUT" -e "RUNNER_OS" -e "RUNNER_ARCH" -e "RUNNER_NAME" -e "RUNNER_ENVIRONMENT" -e "RUNNER_TOOL_CACHE" -e "RUNNER_TEMP" -e "RUNNER_WORKSPACE" -e "ACTIONS_RUNTIME_URL" -e "ACTIONS_RUNTIME_TOKEN" -e "ACTIONS_CACHE_URL" -e "ACTIONS_RESULTS_URL" -e "ACTIONS_ORCHESTRATION_ID" -e GITHUB_ACTIONS=true -e CI=true -v "/var/run/docker.sock":"/var/run/docker.sock" -v "/home/runner/work/_temp":"/github/runner_temp" -v "/home/runner/work/_temp/_github_home":"/github/home" -v "/home/runner/work/_temp/_github_workflow":"/github/workflow" -v "/home/runner/work/_temp/_runner_file_commands":"/github/file_commands" -v "/home/runner/work/projecto_DevOps/projecto_DevOps":"/github/workspace" snyk/snyk:node "snyk" "test" "--file=backend/package.json"

9

10 Testing /github/workspace...

11

12 Tested 89 dependencies for known issues, found 5 issues, 11 vulnerable paths.

13

14

15 Issues to fix by upgrading:

16

17 Upgrade body-parser@1.20.3 to body-parser@1.20.5 to fix

18 x NULL Pointer Dereference (new) [Medium Severity][https://security.snyk.io/vuln/SNYK-JS-QS-16721866] in qs@6.13.0

19 introduced by body-parser@1.20.3 > qs@6.13.0 and 2 other path(s)

Análisis de Código Estático (SonarCloud)

El proyecto ha sido evaluado en la nube arrojando las métricas de calidad de software correspondientes.

projecto_DevOps

Project

Overview

Analysis

Summary

Issues

Architecture

Security hotspots

Reporting

Measures

Activity

Policies

Intended architecture

Quality profiles

Quality gate

Project

Pull Requests

Branches

Code

Project Information

Andres Santiago Santafe Silva / proyecto_DevOps / Overview

Overview

Public • No tags • 784 Lines of Code • Last analysis 2 minutes ago

View on GitHub

Quality Gate Status

Failed

1 condition failed

27

Open Issues

Overall code - Status: Open

27

Duplications

Overall code

0.0%

No change vs last 30 days

Coverage

Overall code

No data available to display

Security snapshot

Security Rating

Overall code

C

Security Issues

Overall code - Software quality: Secu...

12

Open Security Issues by Severity

Overall code - Software quality: Security - Slice by: Severity

Low 17%

Medium 83%

Reporte de Vulnerabilidades de Dependencias (Snyk)

Resultados obtenidos directamente del escaneo del gestor de paquetes del proyecto:

Tested 89 dependencies for known issues, found 5 issues, 11 vulnerable paths.

– Mongoose (Severidad Alta): Vulnerabilidad de Inyección detectada en la

v8.10.0.

– Express (Severidad Alta/Media): Errores de asignación de recursos sin límites a través de la librería qs.

The screenshot shows the Snyk dashboard for the tenant 'santiagosantafe'. The left sidebar contains navigation links for Tenant, Group, Organization, Dashboard, Projects, Integrations, Members, and Settings. The main content area displays two sections: 'Top pending tasks' and 'Top vulnerable projects'.

Top pending tasks

Project	Fixable Issues	Actions
SantiagoSantafe/proyecto_DevOps	174 C, 410 H, 288 M, 751 L	Fix vulnerabilities
SantiagoSantafe/proyecto_DevOps	0 C, 3 H, 2 M, 0 L	Fix vulnerabilities

Showing 1-2 of 2

Top vulnerable projects

Project	Tested	Issues	Actions
SantiagoSantafe/proyecto_DevOps:backend/Dockerfile	13 minutes ago	24 C, 87 H, 55 M, 447 L	Fix vulnerabilities
SantiagoSantafe/proyecto_DevOps:backend/package.json	13 minutes ago	0 C, 3 H, 2 M, 0 L	Fix vulnerabilities
SantiagoSantafe/proyecto_DevOps:frontend/Dockerfile	13 minutes ago	0 C, 1 H, 0 M, 15 L	Fix vulnerabilities
SantiagoSantafe/proyecto_DevOps:backend/api-deployment.yaml	3 minutes ago	0 C, 0 H, 4 M, 4 L	
SantiagoSantafe/proyecto_DevOps:frontend/frontend-deployment.yaml	3 minutes ago	0 C, 0 H, 3 M, 6 L	

[View all projects](#) → Showing 1-5 of 9

2. Evidencias de Entrega Continua (CD)

Captura del Pipeline de Jenkins (Stage View)

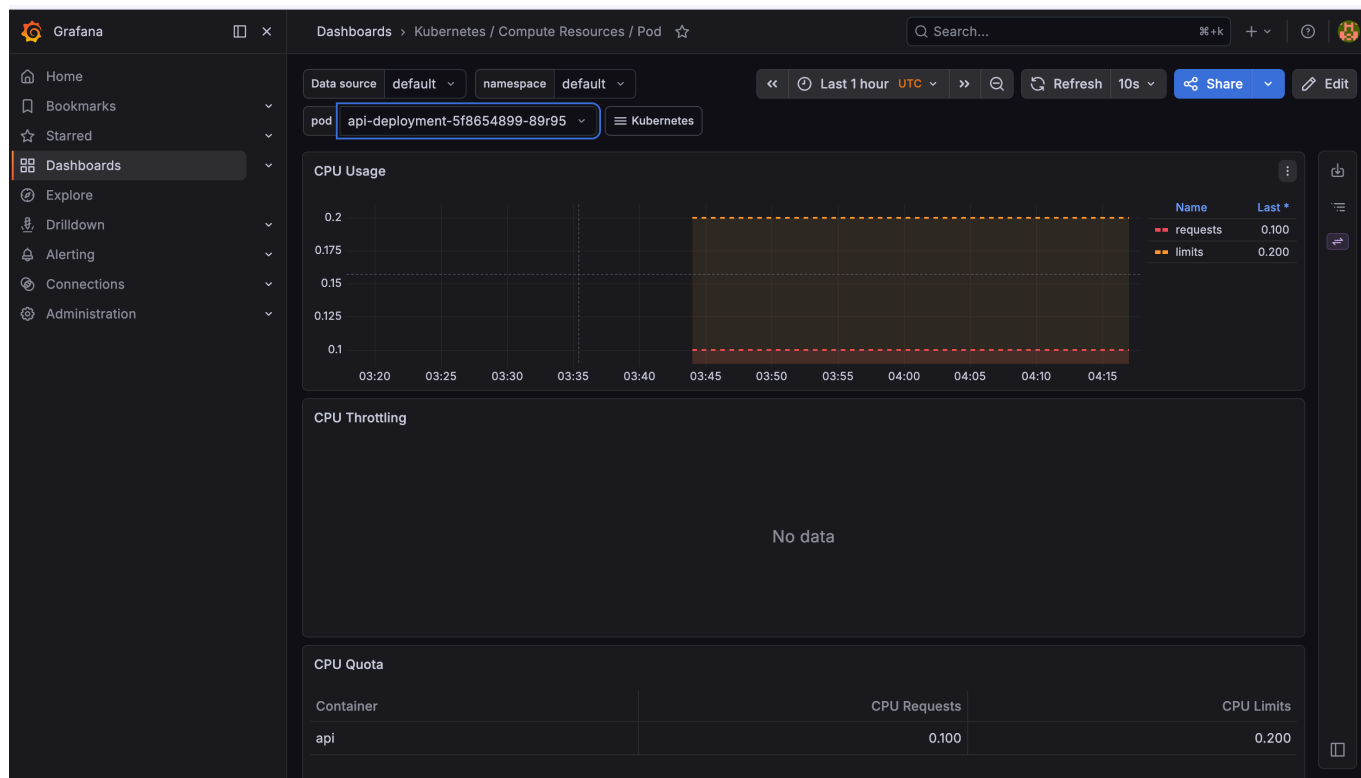
Evidencia visual de la ejecución organizada de todas las etapas operativas del pipeline de CD en bloques verdes secuenciales.

The screenshot shows the Jenkins pipeline stage view for the project 'proyecto_DevOps'. The 'Stages' section displays a successful build on June 10, 2026, at 23:07 - 1s. The build status is indicated by a green checkmark and a sequence of green circles representing the stages. A 'Build' button is visible in the top right corner.

3. Evidencias de Monitoreo Activo (Grafana)

Dashboard de Recursos de Kubernetes (Pods de la Aplicación)

Visualización activa y en tiempo real de las métricas clave extraídas por Prometheus desde el clúster gestionado en OrbStack. Se evidencia el control sobre los límites de CPU y memoria del pod de la API.



Informe de Seguridad y Recomendaciones de Mejora

Basado en los hallazgos críticos reportados por las herramientas integradas en nuestro ecosistema, se determina el siguiente plan de acción inmediato para el equipo de desarrollo:

- Mitigación en Base de Datos:** Se requiere actualizar de forma obligatoria la librería **mongoose** de la versión **8.10.0** a la **8.22.1** para neutralizar por completo el riesgo de inyecciones de código hacia MongoDB.
- Remediación del Servidor HTTP:** Actualizar **express** a la versión **4.22.2** y **body-parser** a la **1.20.5**. Con esto se mitigan las vulnerabilidades de Denegación de Servicio (DoS y ReDoS) provocadas por el manejo ineficiente de expresiones regulares y asignación de memoria del componente interno **qs**.
- Seguridad en Código:** Analizar los 5 *Security Hotspots* indicados por SonarCloud para verificar que no existan credenciales hardcoded o configuraciones de CORS expuestas de manera permisiva en el archivo **server.js**.

Reflexión sobre la Eficiencia Operativa

La transición hacia una cultura **DevSecOps** mediante este laboratorio demuestra que la automatización de la seguridad y el monitoreo no añade burocracia, sino velocidad y resiliencia:

- Seguridad Preventiva (Shift Left):** Al integrar Snyk y SonarCloud en el flujo inicial de GitHub Actions, las vulnerabilidades son interceptadas en la máquina del desarrollador mucho antes de llegar a producción. Esto reduce drásticamente el costo de reparación de fallos.
- Observabilidad de Negocio:** La integración transparente de Prometheus y Grafana mediante Helm dota al equipo de infraestructura de telemetría en tiempo real. Esto permite realizar escalados automáticos basados en datos (HPA) y detectar fugas de memoria o saturación de CPU antes de que afecten la experiencia del usuario final.

- **Consistencia:** Jenkins elimina el factor del error humano en los despliegues de Kubernetes, garantizando que el entorno productivo sea siempre un reflejo exacto y validado de lo que reside en el repositorio de código.